THE CRONOS FRAMEWORK

A Strategic Methodology for Human-Validated Vibe Coding and Agentic Software Engineering

Ovidiu M. M. G

Version 2.1 | 2026

Abstract

The software development landscape of 2025 and 2026 has been fundamentally reshaped by the emergence of vibe coding — a paradigm shift where the primary role of the developer transitions from manual code construction to high-level intent orchestration. Coined by Andrej Karpathy in early 2025, vibe coding describes a workflow where engineers interact with large language models and autonomous agents through natural language to generate, refine, and deploy applications.

While this transition offers unprecedented speed, it introduces systemic risks concerning architectural integrity, long-term maintainability, and security. The Cronos methodology is a formal response to these challenges, establishing a structured framework that preserves generative velocity while embedding rigorous human-centric validation and project management controls.

By organising development into distinct one-week cycles fortified by mandatory peer review and deterministic requirement engineering, Cronos ensures that the creative speed of AI-driven generation is always balanced by the rigour of professional engineering standards. Version 2.0 of this document extends the original framework with universal scalability guidance — covering team size adaptation, remote and asynchronous operation, regulated-industry compliance layers, tooling tiers, a Cycle 0 onboarding protocol, a structured retrospective process, and multi-developer cycle management — making Cronos applicable to any team, project type, or budget.

Keywords: Vibe Coding, Agentic Software Engineering, SDLC, Human-in-the-Loop Validation, AI-Augmented Development, Requirement Engineering, Sprint Cadence

The Strategic Foundations of the Cronos Methodology

The genesis of Cronos lies in the recognition that traditional Agile methodologies, while effective for human coordination, often fail to keep pace with the collapsed feedback loops of AI-driven development. In the pre-AI era, software development cycles were constrained by the human speed of writing logic, necessitating long sprint cycles and extensive ceremonies to manage handoffs.

In the current agentic era — where tools like Cursor, Replit Agent, and Google Antigravity can scaffold entire microservices in minutes — the bottleneck has shifted from implementation to validation and strategic alignment. Cronos addresses this by reconfiguring the software development lifecycle (SDLC) around the concept of 'Cycles', where each cycle is a one-week burst of high-intensity, AI-augmented creation coupled with a fixed duration of human oversight.

Research indicates that while AI can make developers 50x to 100x faster on highly patterned tasks, the overall gain across a project lifecycle typically settles between 7x and 10x due to the complexities of debugging and architectural integration. Cronos is designed to maximise these gains by institutionalising the Vibe-to-Verify Ratio — a metric used to calibrate the level of human scrutiny based on the risk profile of the component being built.

Definition — The Vibe-to-Verify Ratio (VtV)

VtV = T_vibe / T_validation

Where T_vibe is the total time spent in the AI generation loop (prompting, scaffolding, iterative refinement) during a cycle, and T_validation is the total time spent in human-controlled quality gates: peer review, automated QA sign-off, security scan review, and the Thursday QA Sync.

A VtV above 8:1 signals that validation is under-resourced for the feature's complexity. A VtV below 2:1 signals that the AI generation phase may be under-utilised — review whether the 72-Hour Modularisation Mandate is being applied too conservatively.

Target range: 3:1 to 6:1 for standard features; 2:1 to 4:1 for security-sensitive or regulated features.

The Peer Reviewer records the cycle's actual VtV ratio during the Friday Demo as an input to the next cycle's Kickoff planning.

Cronos vs. Traditional Agile: Key Differences

Development Metric	Traditional Agile (2020–2024)	Cronos Methodology (2025–2026)
Core Planning Unit	2-Week Sprint	1-Week Cycle
Primary Coding Mode	Manual / Line-by-line	Vibe Coding / Agentic
Validation Cadence	End of Sprint	Tue / Thu / Fri Syncs
Documentation Strategy	Post-hoc / Manual	Real-time / AI-Generated
Release Velocity	Bi-weekly / Monthly	Weekly / Continuous
Role of Developer	Creator / Author	Curator / Solution Governor

PART A — UNIVERSAL SCALABILITY EXTENSIONS

The sections below represent Version 2.0 additions to the original Cronos specification. They address the framework's applicability across different team sizes, geographic distributions, budget constraints, and regulatory environments.

A.1 Team Size Scaling Model

The original Cronos specification was implicitly designed around a small, co-located team of three to four people. In practice, organisations vary from solo developers to large enterprise pods. The following scaling model defines how roles collapse or expand across team sizes, and which ceremonial events remain mandatory at each tier.

Tier 1 — Solo Developer

Role Mapping

The developer assumes all four roles simultaneously: SO, TPO, Developer, and Peer Reviewer.

Peer review is replaced by a structured self-review using the full Peer Validation Checklist, ideally performed after a minimum 4-hour gap from the last code commit (cold-read effect).

The Tuesday Path Sync becomes a personal written status note committed to the repository.

The Friday Demo becomes a recorded Loom / async video shared with any stakeholder.

- Mandatory events: Thursday QA Sync (self-audit), Friday Demo (async recording).
- Recommended: use a second AI session in a separate context window as a simulated peer reviewer.
- The 72-Hour Modularisation Mandate remains fully in force.

Tier 2 — 2–4 Person Team (Startup / Small Studio)

Role Mapping

SO and TPO roles may be collapsed into one person who is not the primary Developer.

Peer Reviewer is any team member who did not author the cycle's code.

All four core syncs (Kickoff, Tuesday, Thursday, Friday) are held live — typically 15–30 min each.

Complex Project Protocol activates when the Epic touches more than two domains (e.g., auth + payments + third-party API).

- All original Cronos ceremonies are mandatory at this tier.
- Multi-Track (Parallel Feature Track) is viable from two developers.

Tier 3 — 5–10 Person Product Team

Role Mapping

SO, TPO, Developer(s), and Peer Reviewer are distinct individuals.

Multiple developers may run parallel cycles on separate Epics simultaneously (Multi-Track).

A dedicated QA engineer joins Thursday's Checkpoint & QA Sync.

The Friday Demo is presented to a wider stakeholder group, not just the internal team.

- Introduce a Cycle Board (Kanban-style) to track parallel cycles visually.
- Establish a shared .cursorrules or copilot-instructions.md repository used by all developers.
- Rotate the Peer Reviewer role across developers to distribute knowledge.

Tier 4 — 10+ Person Enterprise Pod

Role Mapping

A dedicated Solution Owner per product area; a single TPO or Architecture Council governs cross-cutting decisions.

Developers are grouped into 'Cycle Pods' of 2–3, each running its own Cronos cycle.

A Release Train Engineer (RTE) coordinates Friday Demos across pods to manage integration dependencies.

Security and compliance officers join Thursday's QA Sync for regulated features.

- Mandate architectural decision records (ADRs) for all cross-pod dependencies.
- Enterprise tooling budget unlocks Tier 1 tooling (see Section A.4).
- Consider quarterly 'Cronos Health Reviews' to assess framework adherence across pods.

A.2 Remote and Asynchronous Operation

The original Cronos rhythm assumes synchronous, co-located participation. For distributed teams operating across time zones, the following async-compatible variants replace or supplement each synchronous touchpoint while preserving its validation intent.

Sync Event	Sync (Default)	Async Variant	Max Async Delay
Requirements Presentation	Live meeting (60 min)	Recorded Loom + written PRD review thread	48 hours before Kickoff
Monday Kickoff	Live standup (30 min)	Written kickoff note in project hub + async Q&A window	End of Monday

Tuesday Path Sync	Live call (15–30 min)	Video walkthrough + comment thread on PR / task	Tuesday EOD
Thursday QA Sync	Live review (30–60 min)	Shared testing report + async annotation session	Thursday EOD
Friday Demo	Live demo (30–60 min)	Recorded demo + async feedback form (48 h window)	Sunday EOD

Async Operation Rules

- Every async deliverable must be timestamped and stored in a shared, version-controlled location (not email or ephemeral chat).
- A 'Async Approval Window' of 24 hours is granted after each async deliverable. Silence after this window is treated as approval. Objections must be logged as written comments.
- The Thursday QA Sync may never be fully replaced by async alone for security-sensitive features — at minimum, a synchronous 15-minute call is required.
- Reset Triggers (Section IV) apply regardless of sync mode. An Emergency Sync Meeting triggered by a Reset Trigger must be held synchronously, even if it requires scheduling across time zones within 4 hours of the trigger event.

A.3 Regulated Industry Compliance Layer

Healthcare, fintech, legal, and other regulated sectors impose requirements that the base Cronos cycle does not address by default. This section defines a Compliance Overlay — a set of additional checkpoints and artefacts that are appended to the standard cycle without altering its core rhythm.

Supported Regulatory Frameworks

- HIPAA / HITECH (Healthcare — United States)
- SOC 2 Type II (General SaaS / Cloud Services)
- GDPR / ePrivacy (European Union — Data Privacy)
- PCI-DSS (Payment Card Industry)
- ADA / WCAG 2.2 AA (Accessibility — US Public-Facing Web)
- FDA 21 CFR Part 11 (Healthcare Software / Electronic Records)

Compliance Overlay Additions per Cycle

Cycle Phase	Compliance Addition	Applicable Frameworks

Phase 0 — Requirements	PRD must include a Data Classification section (PII, PHI, PCI scope)	HIPAA, GDPR, PCI-DSS
Monday — Init	Threat model sketch committed alongside scaffolding	SOC 2, HIPAA
Thursday — QA	Automated accessibility scan (axe-core / Deque) as required check	ADA / WCAG 2.2 AA
Thursday — QA	SAST scan results reviewed and signed off by TPO	SOC 2, PCI-DSS
Friday — Demo	Change record created in audit log before deployment	SOC 2, FDA 21 CFR
Hypercare	Extended to 7 days (not 48 hours) for patient-facing features	HIPAA, FDA 21 CFR
Post-Cycle	ADR (Architectural Decision Record) filed for any data-layer change	GDPR, SOC 2

Extended Hypercare for Regulated Features

Standard Hypercare: 48 hours post-release observation.

Regulated Hypercare: Minimum 7 days for features handling PHI, PII, or payment data.

During Regulated Hypercare, the SO must review telemetry dashboards daily and log observations.

Any anomaly triggering an alert must be escalated within 4 hours, regardless of business hours.

Rollback procedures must be documented and tested before the Friday Demo concludes.

A.4 Tooling-Agnostic Tier Model

The original framework referenced premium commercial tooling throughout. The following tier model ensures Cronos is accessible to teams with any budget. Every Cronos phase can be executed at each tier — only the specific tool changes.

Phase	Tier 1 — Premium	Tier 2 — Mid-Range	Tier 3 — Free / OSS
AI Coding Assistant	Cursor Pro, GitHub Copilot Enterprise	GitHub Copilot (Individual), Codeium	Continue.dev, Ollama + CodeLlama
Scaffolding / Agents	Google Antigravity, Replit Agent	v0.dev, Bolt.new	Aider, OpenHands (OSS)
PRD & WBS	ChatPRD, Linear AI	Notion AI, ClickUp AI	GitHub Issues + AI prompt templates
Automated Testing	Sauce Labs AI, QA Wolf	Playwright (cloud), BrowserStack	Playwright (local), Vitest, Jest
Visual QA	Applitools, Percy	Chromatic (OSS tier)	Percy OSS, BackstopJS

Security Scanning	Checkmarx One, Snyk Team	Semgrep OSS + CI, GitHub Advanced Security	Semgrep OSS, Bandit, ESLint Security
Documentation	Mintlify, GitBook Pro	Docusaurus + AI	MkDocs, README.md + AI
Project Tracking	Monday.com, Jira Advanced	Linear, Shortcut	GitHub Projects, Plane (OSS)
Meeting AI	Fireflies.ai, Otter.ai	Otter (Basic), tldv (Free)	Local whisper transcription

Tier Selection Guidance

Teams should select a consistent tier across all phases — mixing Tier 1 and Tier 3 tools creates integration gaps.

Tier 3 teams must allocate approximately 20% additional developer time per cycle to compensate for reduced automation.

Any tool can be upgraded between cycles. Tool changes must be documented in the Kickoff for the next cycle.

Open-source tools at Tier 3 are fully Cronos-compliant — the methodology does not require commercial tooling.

A.5 Cycle 0 — Onboarding Protocol

The original framework assumed that infrastructure, prompt libraries, and team calibration already existed. In practice, every new project or new team requires a foundational 'Cycle 0' before the first production cycle begins. Cycle 0 is a one-time investment, not a recurring event.

Cycle 0 Activities (One-Time, Pre-Cycle 1)

Activity	Owner	Output / Artefact
Toolchain Setup	Developer + TPO	Verified dev environment, CI/CD pipeline, repo structure
AI Guardrails Config	Developer	.cursorrules / copilot-instructions.md committed to repo
Prompt Library Seeding	Developer + SO	Initial library of 10–20 engineered prompts for common patterns
Architectural Blueprint	TPO	Tech stack doc: languages, frameworks, versions, patterns
Velocity Baseline	SO + Developer	Estimated cycle capacity (hours/week) per developer role

Compliance Checklist	TPO + SO	Regulatory requirements identified; Compliance Overlay activated if needed
Communication Norms	SO	Sync vs. async decision; tools for each event; timezone agreements
Peer Review Rotation	SO	Documented schedule of who reviews whom across first 4 cycles
Definition of Done	SO + TPO	Project-level acceptance criteria appended to PRD template
Risk Register	SO	Top 5 architectural unknowns documented with mitigation notes

Cycle 0 is considered complete when all artefacts above are committed to the repository and reviewed by the full team. Only after this gate may Cycle 1 begin.

Cycle 0 Time Estimate by Tier

Tier 1 (Solo): 4–8 hours total.

Tier 2 (2–4 persons): 1 day total, ideally a dedicated 'Cronos Setup Day'.

Tier 3 (5–10 persons): 1–2 days, including a full team kick-off workshop.

Tier 4 (10+ persons, Enterprise): 1 week, treated as a formal Sprint 0 with its own artefacts.

A.6 Cycle Retrospective — The 'Cronos Autopsy' Protocol

The original framework defined Reset Triggers for mid-cycle failures but provided no structured retrospective for completed cycles. The Cycle Autopsy is a lightweight, time-boxed post-cycle retrospective that feeds directly into the next cycle's PRD and Kickoff.

When to Hold a Cycle Autopsy

- After every cycle where a Reset Trigger was activated.
- After any cycle that missed its Friday Demo deliverable.
- After any cycle where the Peer Reviewer raised more than three architectural concerns.
- Optionally, after every third cycle as a proactive health check (recommended for Tier 3 and 4 teams).

Autopsy Format (Time-Boxed: 45 Minutes Maximum)

Segment	Duration	Facilitator	Output
What shipped?	5 min	SO	Confirmed deliverables vs. planned

What broke the vibe?	10 min	Developer	Root cause of stagnation or drift
What did the AI get wrong?	10 min	Peer Reviewer	Hallucination log: APIs, logic, security
What do we fix in the PRD?	10 min	SO + TPO	Delta requirements for next cycle
Action items	10 min	All	Max 3 written actions with owners and deadlines

The Hallucination Log — Formal Definition

The Hallucination Log is a persistent, shared artefact (repository wiki or project hub document) that accumulates across all cycles for the lifetime of a project. It is not an optional retrospective output — it is a required deliverable of every Cronos Autopsy and of every Reset Trigger 2 (Circular Hallucination) event.

Field	Description	Example Entry
Cycle	Cycle number in which the hallucination occurred	Cycle 4
AI Tool	The specific model or agent that produced the hallucination	Cursor / claude-sonnet-4-6
Hallucination Type	Category: Non-existent API | Incorrect Logic | Insecure Pattern | Wrong Dependency | State Error	Non-existent API
Description	Exact nature of the hallucination — what the AI claimed vs. what was true	Called stripe.charges.createIntent() — method does not exist in Stripe SDK v14
Detection Method	How it was caught: Peer Review | Automated Scan | Thursday QA | Production Bug | Developer (mid-loop)	Peer Review — Friday Demo
Mitigation Applied	The fix implemented and the guardrail added to prevent recurrence	Added Stripe SDK method whitelist to .cursorrules; added integration test asserting valid method names
Severity	Low | Medium | High | Critical (based on whether it reached production)	High

Over time, the Hallucination Log's Mitigation column directly feeds the team's .cursorrules and prompt library. A log with 20+ entries across 10 cycles represents a significant competitive asset — a hardened, project-specific AI guardrail system that no off-the-shelf tool can replicate.

A.7 Multi-Developer Cycle Management

The original framework assumed a single developer per cycle. When two or more developers share a cycle, ownership boundaries and merge strategies must be defined explicitly to prevent conflicts, duplicated effort, and context fragmentation.

Splitting an Epic Across Developers

- The Epic is divided into 'Feature Tracks' at the Kickoff meeting. Each track is a self-contained module with clearly defined interfaces (API contracts, shared type definitions).
- Each developer owns exactly one Feature Track per cycle. Ownership is exclusive — no track may be touched by the non-owner developer without explicit handoff.
- Shared state (database schema, authentication layer, shared utilities) is owned by the senior developer or TPO and is considered frozen during the cycle unless an Emergency Sync is called.
- Feature Tracks are developed on separate Git branches. Branches merge into a shared integration branch (not main) on Thursday for joint QA.

Multi-Developer Sync Cadence

Event	Additional Step for Multi-Developer Cycles
Monday Kickoff	Feature Track assignment documented; interface contracts committed
Tuesday Path Sync	Each developer gives a 5-minute status; interface conflicts flagged immediately
Wednesday (Mid-loop)	30-min cross-developer integration check — confirm branches will merge cleanly
Thursday QA Sync	Merged integration branch tested jointly; TPO owns conflict resolution
Friday Demo	Both developers present their Feature Track; Peer Reviewer reviews both tracks

Conflict Resolution Protocol

- If two Feature Tracks require conflicting changes to a shared interface, the TPO has unilateral authority to resolve the conflict.
- If a developer falls behind their Feature Track, the SO may reassign tasks from their track to the other developer — but may not split a single task across developers.
- The Circular Hallucination Reset Trigger applies per developer independently — if one developer hits it, an Emergency Sync is called for the whole cycle, not just that track.

PART B — CORE FRAMEWORK SPECIFICATION

I Defined Roles and Responsibilities

The following roles form the minimum viable team for a Cronos cycle. Refer to Section A.1 for how roles collapse or expand by team size.

Role	Primary Responsibility	Critical Presence
Solution Owner (SO)	Merged PM/PO role; crafts agent-ready PRDs, manages the WBS, and holds final release authority. Acts as Supervisor/Peer Programmer for complex builds. Lead for Phase VI Observation.	All Events (Reqs, Kickoff, Tue/Thu Syncs, Friday Demo)
Technical Product Owner (TPO)	Validates technical constraints in PRDs, ensures architectural blueprints are followed, and bridges business needs with technical reality.	Requirements Presentation, Thursday QA Sync, and Friday Demo
Developer	Intent orchestration, vibe coding execution, and acting as 'Mission Control' for AI agents.	All technical events and daily cycle tasks
Peer Reviewer	Provides 'Extra Human Validation' and executes the survivability checklist to prevent author bias.	Requirements Presentation and Friday Demo

II Complex Project Protocol

For initiatives requiring high feature density, the Solution Owner is mandated to maintain a direct presence during the development cycle. In these cases, the SO transitions from a passive stakeholder to an active supervisor or 'Prompt-Level Peer Programmer', ensuring real-time alignment with business logic as the AI generates code.

III The Strict 7-Day Cronos Rhythm

The schedule is deterministic. If a cycle is completed early, the sequence of events is accelerated, but no stage is skipped.

1. The Focus Mandate (The 'Flow Shield')

To preserve the exponential efficiency of agentic workflows, the developer assigned to the cycle must have their calendar 100% blocked. No external meetings are permitted except for the

methodology-specified syncs: Kickoff, Tuesday Path Sync, Thursday QA Sync, and Friday Demo.

2. The 72-Hour Modularisation Mandate

If a feature or Epic is estimated to require more than 3 days of active development (Monday Initialisation through Wednesday Vibe Coding), it must be modularised. The project must be broken into functional 'chunks' that fit within a 3-day implementation window. This prevents 'Vibe Drift' and ensures Thursday's Rigorous Verification remains comprehensive.

Phase 0 — The Foundation (Previous Friday)

- Requirements Presentation Meeting: The SO presents the agent-ready PRD. The TPO validates technical feasibility, the Developer assesses 'vibe-ability', and the Peer Reviewer begins mental model preparation.
- Attendees: SO, TPO, Developer, Peer Reviewer.

The Development Cycle — Monday through Friday

Day / Phase	Activity	Owner(s)	Goal
Monday — Initialisation	Scaffold project; set .cursorrules or environment instructions	Developer	Foundation set and plan approved
Tuesday AM — Path Sync	15–30 min: ensure project is on the right path with zero logic doubts	SO + Developer	Alignment confirmed
Tue–Wed — High-Vibe Execution	'See stuff, say stuff, run stuff' loop with AI. PO acts as supervisor on complex projects.	Developer (+ SO for complex)	Features built, vibe achieved
Thursday AM — QA Sync	Verify checkpoint. SO, TPO, Developer (+ optional QA) review and build test suites.	SO + TPO + Developer	Functional and security integrity
Thursday — Rigorous Verification	Automated testing, security scans (ReDoS, insecure patterns), visual audits	Developer + QA	All acceptance criteria met
Friday Midday — Demo & Review	Formal demonstration. SO, TPO, and Peer Reviewer conduct final validation.	SO + TPO + Peer Reviewer	Release decision made
Friday PM — Polish & Release	AI-generated documentation sync; deployment to production-ready environments	Developer	Releasable artefact delivered

Post-Release — Hypercare	48-hour (standard) or 7-day (regulated) observation: telemetry, logs, user feedback	SO + QA	Production stability confirmed

IV Timeline Governance and Reset Triggers

The Compression Protocol

If the developer finishes the build ahead of schedule, the Thursday and Friday events must be moved forward immediately. The Vibe-to-Verify ratio remains constant — speed does not excuse the bypass of the TPO or Peer Reviewer checkpoints.

Mandatory Reset Triggers

If development hits any of the following scenarios, an Emergency Sync Meeting with the Solution Owner is automatically triggered. During that meeting the SO and Developer set new, strict timelines while AI project management agents identify blockers and suggest resource reallocation.

Reset Trigger 1 — 4-Hour Prompt Loop Stagnation

If a developer spends more than 4 consecutive hours in a conversational loop with the AI agent without achieving a functional state or a code commit for a specific sub-task, an Emergency Sync is triggered immediately.

Reset Trigger 2 — Circular Hallucination Threshold

If an AI agent proposes the same failing solution or logic structure more than three times despite the developer providing different context or corrective prompts, a Reset is triggered and the hallucination is logged in the Hallucination Log.

Reset Trigger 3 — 72-Hour Chunk Breach

If, by the end of Tuesday, the developer identifies that the modularised 72-hour chunk contains architectural 'unknown-unknowns' (hidden dependencies) that will prevent a functional demo by Friday Midday, a Reset is triggered.

Reset Trigger 4 — Tool-Chain Interruption

Any failure or outage of critical agentic infrastructure (e.g., Cursor, Google Antigravity, or deployment orchestration) lasting more than 2 hours during the High-Vibe phase triggers an Emergency Sync.

Phase I Requirement Engineering and the Solution Owner's Mandate

In the Cronos framework, the Solution Owner assumes a more technical and deterministic role in the early stages of the project. The traditional ambiguity of requirements — often left for developers to interpret during implementation — is a primary cause of hallucination and rework in AI-driven projects. Cronos mandates that the SO gathers requirements and distils them into a Product Requirement Document (PRD) that is specifically engineered for AI agents.

From Human Interpretation to Agent-Ready Determinism

The SO's responsibility is to translate stakeholder needs into a specification that leaves zero room for the AI to 'guess'. This involves moving beyond descriptive goals toward deterministic constraints. For example, instead of requesting a 'responsive dashboard', the SO must specify the exact state machine the dashboard will inhabit, including idle, loading, error, and success states.

PRD Component Reference

PRD Component	Role in Cronos	Impact on Agentic Execution
State Enumeration	Define all possible UI / System states	Eliminates guessing in UI generation; ensures edge case coverage
API Contracts	Specify request / response schemas	Prevents integration errors between frontend and backend agents
Tech Constraints	List mandatory libraries and versions	Ensures code follows security and performance standards
Acceptance Criteria	Per-requirement testable conditions	Provides the AI with a Definition of Done for automated verification
Data Classification	Label PII, PHI, PCI scope (regulated overlay)	Ensures compliance checks are targeted and proportionate

Phase II The Kickoff and Collaborative Estimation

The transition from the requirement phase to active development is marked by the Kickoff meeting — a critical synchronisation point where the SO explains the Epics to the developers. Unlike traditional sprint planning focused on story points, the Cronos Kickoff focuses on Cycles.

Developers review the Epics and the SO's documentation to determine how many one-week cycles are necessary. The estimation process is increasingly supported by AI project management agents that analyse historical project data to predict effort, cost, and timelines. The

developer's primary input during this meeting is architectural: identifying potential 'hot spots' where the AI might struggle.

Establishing Governance and Coding Rules

The Kickoff also establishes the 'Rules of Engagement' for the AI. Developers configure environment-level instructions such as .cursorrules or copilot-instructions.md, which serve as guardrails for the vibe coding process. These files contain coding style preferences, security policies (e.g., 'Never use eval()'), and architectural patterns (e.g., 'Use repository pattern for data access').

Phase III The One-Week Cronos Cycle

The core of the Cronos methodology is the one-week cycle — a rhythmic unit of development that integrates generation, verification, and documentation into a continuous flow. This structure prevents the 'vibe coding trap' where developers focus exclusively on feature creation for weeks, only to find themselves buried under technical debt at the end of the project.

Cycle Phase	Activity	Duration	Goal
Initialisation	Prompting / Scaffolding	0.5–1 Day	Foundation set and plan approved
Implementation	Vibe Coding Iterations	2 Days	Features built and vibe achieved
Verification	Automated Testing / QA	1 Day	Functional and security integrity
Validation	Peer Review (HITL)	0.5 Day	Human accountability and logic check
Finalisation	Docs / Polish / Prep	0.5 Day	Releasable state and documentation

Phase IV The Extra Human Validation Constraint

The most critical innovation of the Cronos framework is the mandate for 'extra human validation' — specifically allocating a half-day of review by someone other than the original developer. This practice addresses the 'Author's Bias' inherent in vibe coding, where the person who prompted the AI becomes over-reliant on its confident-sounding output and fails to see subtle logic errors.

Data from security audits show that teams using a specialised AI-security checklist caught 94% of vulnerabilities in review, compared to only 62% for those using traditional processes. The

47% increase in review time is more than offset by the reduction in production bug fixes, which are historically 10x to 100x more expensive.

In traditional environments, code review is often a cursory syntax and style check. In Cronos, the half-day validation is a 'survivability requirement'. The reviewer mentally executes the code as an attacker would — asking uncomfortable questions about what happens when inputs arrive out of order, when state is manipulated directly, or when a dependency silently changes its API contract.

Peer Validation Checklists for Cronos

The following checklists are mandatory artefacts of the half-day review. The Peer Reviewer works through every row systematically. Any row that cannot be answered confidently with 'yes' or 'N/A' constitutes a blocking concern that must be documented and resolved before the Friday Demo release decision is made.

Review Category	Logic and Architecture Questions	Security and Integrity Questions
Hallucination Check	Does every import actually exist in our installed environment? Are all called methods real methods of that library version?	Are we calling libraries that are actively maintained and free of known CVEs?
Auth & Identity	Is authorisation logic enforced server-side, not just in UI state or conditional rendering?	Can a user bypass the UI layer entirely and call this API endpoint directly with a crafted request?
State & Data	How does the system handle concurrent saves from two sessions? Does the AI assume optimistic locking where none exists?	Is sensitive data (tokens, PII, session state) stored only in httpOnly cookies or server-side — never in localStorage?
Edge Cases	What happens if a network call fails mid-transaction? Is partial state rolled back or left inconsistent?	Is there explicit input validation for null, negative, empty string, and boundary values — not just 'happy path' inputs?
Dependencies	Are all third-party package versions pinned? Did the AI introduce any package not in the approved dependency list?	Has a dependency licence audit been run? Are any packages under a licence incompatible with the project's distribution model?
Observability	Does every significant operation produce a log entry? Will the team be able to reconstruct what happened from logs alone if a production issue occurs?	Are error messages sanitised before being returned to the client? Does any error path leak stack traces or internal paths?

Phase V The Review Meeting and Release Readiness

At the end of the one-week cycle, a Review meeting presents and releases the product or its releasable part. In the Cronos framework, the Review meeting is centred on working software, not slides. Because vibe coding allows for the creation of functional prototypes in hours, SOs can present real, production-ready UI to stakeholders every week.

The Review meeting concludes with a decision on release readiness. Even if a feature is functionally complete, the technical lead may recommend withholding it if the Vibe-to-Verify ratio was too low or if the half-day validation revealed architectural concerns requiring refactoring. Cronos uses Change Control features in orchestration platforms to ensure that only approved, validated versions of the code reach the live environment.

VI Maintenance vs. Feature Tracks

Cronos separates the high-vibe feature track from legacy stability work. Standard bug fixes and support requests are handled via the team's regular support procedures (Maintenance Track). New features may be implemented simultaneously using Cronos cycles by other members of the team (Parallel Feature Track). This Multi-Track approach ensures production stability does not stall innovation velocity.

PART C — EFFICIENCY MODELLING AND LONG-TERM GOVERNANCE

Theoretical Modelling of Cronos Efficiency

The productivity gain P of the Cronos framework can be modelled relative to traditional Agile development by considering the implementation speedup Si and the validation overhead Vh. In scenarios where Si >= 10 (typical for CRUD or boilerplate tasks), the Cronos methodology delivers a significant net gain even with the high fixed cost of human validation.

> **Important — Scope of This Model**
>
> The figures in the table below are estimated from practitioner-reported velocity data collected across industry reports and community studies (2025–2026). They are not the result of controlled academic experiments with randomised cohorts.
>
> These estimates should be treated as directional benchmarks, not universal constants. Actual efficiency gains will vary based on: the team's familiarity with AI tooling, the complexity and novelty of the domain, the quality of the agent-ready PRD, and the team's tier (Solo through Enterprise).
>
> Teams are strongly encouraged to measure their own T_vibe and T_validation values from Cycle 1 onward using the VtV Ratio, and to calibrate these benchmarks against their empirical data. The Cronos community welcomes contributions of real-world efficiency measurements to build a validated dataset over time.

Activity	Traditional (Hours)	Cronos (Hours)	Efficiency Delta
Planning & WBS	6	2–3	2–3.0x
Code Construction	25	4–6	4–6.25x
Testing & QA	6	2	3.0x
Peer Review (HITL)	1	4.5	0.22x (intentional cost)
Documentation	2	0.5	4.0x
Total	40	11–14	2.85–3.64x

This model suggests that a Cronos team can complete the work of a traditional 40-hour week in approximately 11 to 14 hours, effectively allowing for multiple episodes of creation within a single one-week cycle, or enabling a single developer to manage multiple small projects simultaneously with high confidence in quality and security.

Speed Optimisation Strategies

1. Automated WBS and Task Orchestration

Solution Owners can use AI agents to automatically generate a detailed Work Breakdown Structure from a project brief. These agents identify task dependencies, estimate durations using historical team velocity, and suggest the best assignee based on current workload and skill sets — reducing the 4–8 hours SOs typically spend on manual breakdown to mere minutes.

2. Meeting-to-Action Item Pipeline

Kickoff and Review meetings can be recorded and transcribed by AI assistants. These tools extract action items and automatically create them as tasks in the team's project hub, ensuring that every decision made during a meeting is instantly actionable and traceable.

3. Hierarchical AI Model Routing

Simple tasks (boilerplate generation, UI styling) are routed to lightweight, high-speed models, while complex reasoning, architectural design, and security reviews are handled by larger, high-parameter models. This expert teamwork approach reduces latency and ensures the most powerful resources are reserved for the most difficult problems.

4. Meta-Prompting Libraries and Architectural Patterns

Teams should develop a centralised library of engineered prompts for common architectural patterns: authentication flows, data access layers, and error-handling middleware. By using frameworks like CRISPE or PRIME, developers ensure the AI produces consistent, trustable output across different parts of the application.

5. Automated Release Notes and Executive Reporting

AI orchestration layers can monitor task updates and code commits to generate automated status reports tiered for different audiences: detailed task completions for the team, high-level milestones for stakeholders, and one-paragraph summaries with health indicators for executive leadership.

6. Agentic Testing and Self-Healing Infrastructure

Agentic Testing has AI agents take ownership of the entire testing lifecycle — scanning repositories, identifying exposed routes, writing their own test cases, and opening pull requests to fix discovered bugs. In production, self-healing infrastructure uses AI to monitor logs and telemetry in real-time, detecting anomalies and proposing patches to resolve performance issues or security threats.

Long-Term Maintenance and Technical Debt Management

A primary risk of high-velocity vibe coding is the accumulation of 'AI-generated debt' — code that is functionally adequate but architecturally fragile or opaque to future developers. Cronos mitigates this through Refactoring Cycles and Architectural Decision Records.

Proactive Refactoring and Debt Gating

Teams using Cronos are encouraged to allocate every fourth or fifth cycle as a 'Technical Health Cycle'. During this time, the goal is not to build new features but to refactor existing AI-generated code for better modularity, performance, and readability. Tools like CodeRabbit or CodeScene provide real-time project health scoring, flagging areas where the AI's guess-work has created overly complex or tightly coupled logic.

Maintaining Strategic Sovereignty

Cronos emphasises the use of local, code-first editors like Cursor for long-term strategic projects. Unlike browser-based agents that may lead to platform lock-in, local IDEs interact with standard Git repositories, allowing the team to maintain absolute control over their codebase. This ensures that as AI models and tools evolve, the organisation retains its strategic sovereignty and can migrate to different hosting providers or frameworks without being trapped by a proprietary black box.

Conclusion: The Cronos Competitive Advantage

The Cronos methodology represents a balanced, professional approach to the new reality of software development. By institutionalising the Solution Owner's role in requirement engineering, the one-week cycle cadence, and the mandatory half-day human validation, Cronos allows organisations to harness the speed of vibe coding while maintaining the rigorous standards of professional engineering.

Version 2.0 extends this foundation to address the full spectrum of real-world deployment contexts: solo freelancers and enterprise pods, co-located and globally distributed teams, greenfield SaaS products and HIPAA-regulated healthcare platforms, premium toolchains and open-source alternatives. The framework's universal scalability model — anchored by the Cycle 0 protocol, the Cronos Autopsy retrospective, and the Team Size Scaling tiers — ensures that Cronos is not merely a methodology for well-resourced technology companies, but a governance standard accessible to any team building software with AI assistance.

In an economy where knowledge as a service will power the future, the Cronos framework ensures that engineering teams remain the strategic overseers of their software factory, leveraging AI as a powerful pair programmer rather than an uncontrolled code generator. As vibe coding continues to democratise development, Cronos provides the necessary governance and quality gates to ensure that the democratisation of building does not lead to a proliferation of insecurity.

Future research directions include longitudinal measurement of the Vibe-to-Verify Ratio across industry sectors, empirical validation of the Hallucination Log as a hardening mechanism, and

the development of a formal Cronos Maturity Model to assess and guide organisational adoption.

Limitations and Future Research

The efficiency figures cited in this document draw from practitioner reports and emerging industry data (2025–2026) rather than controlled academic experiments. Future peer-reviewed studies should seek to validate the productivity model under controlled conditions, account for domain-specific variance (systems programming vs. web CRUD), and measure the long-term trajectory of AI-generated debt across multi-year projects.

The Cronos framework is a living methodology. As AI coding capabilities, tooling ecosystems, and regulatory landscapes evolve, subsequent versions of this document will be issued. Practitioners are encouraged to contribute empirical findings to the Cronos community to accelerate the framework's maturation.

Works Cited

The following references are formatted in APA 7th edition style. Practitioner sources (blog posts, community forums, vendor documentation, social media) are marked [P]; peer-reviewed or institutional sources are marked [A]. The [P] designation does not imply lesser reliability — it indicates the nature of the peer-review process applied to the source.

1. [A] McKinsey & Company. (2026). How an AI-enabled software product development life cycle will fuel innovation. McKinsey Digital. https://www.mckinsey.com/industries/technology-media-and-telecommunications/our-insights/how-an-ai-enabled-software-product-development-life-cycle-will-fuel-innovation
2. [P] Augment Code. (2026). AI agent workflow implementation guide for dev teams. https://www.augmentcode.com/guides/ai-agent-workflow-implementation-guide
3. [P] Bright Security. (2026). 5 best practices for reviewing and approving AI-generated code. https://brightsec.com/blog/5-best-practices-for-reviewing-and-approving-ai-generated-code/
4. [P] ChatPRD. (2026). Writing PRDs for AI code generation tools in 2026. https://www.chatprd.ai/learn/prd-for-ai-codegen
5. [P] ClaudeAI Reddit Community. (2026). The truth about AI-assisted development efficiency: What 12 days and 520,000 lines of code taught me [Practitioner report]. r/ClaudeAI. https://www.reddit.com/r/ClaudeAI/comments/1s2gu30/
6. [P] Forbes Tech Council. (2025, July 28). 20 expert strategies to optimise AI speed and performance. Forbes. https://www.forbes.com/councils/forbestechcouncil/2025/07/28/20-expert-strategies-to-optimize-ai-speed-and-performance/
7. [P] Graham, O. (2026). Why Agile is dead: My experience with vibe coding. Medium. https://oldane-graham.medium.com/why-agile-is-dead-my-experience-with-vibe-coding-28cf19f7ca97
8. [P] IBM Think. (2026). What is Human in the Loop (HITL)? IBM. https://www.ibm.com/think/topics/human-in-the-loop

9. [P] inapps.net. (2026). Vibe coding vs best practices: When fast code becomes a problem. https://www.inapps.net/vibe-coding-vs-best-practices-when-fast-code-becomes-a-problem/
10. [P] IntuitionLabs. (2026). Agile vs. vibe coding: Why Agile is still essential for AI. https://intuitionlabs.ai/articles/agile-vs-vibe-coding
11. [P] Karpathy, A. (2025, February). Coinage of the term 'vibe coding' [Social media post / public remarks]. Referenced in: Wikipedia contributors. (2026). Vibe coding. Wikipedia. https://en.wikipedia.org/wiki/Vibe_coding
12. [P] Kodus. (2026). AI-generated code requires a different code review process. https://kodus.io/en/ai-generated-code-review-new-processes/
13. [P] Mishra, D. (2026). Cursor vs Replit vs Lovable: The AI coding war that will decide how software gets built. DEV Community. https://dev.to/deepak_mishra_35863517037/cursor-vs-replit-vs-lovable-3fo6
14. [P] Monday.com. (2026). AI for software engineering: Practical implementation strategies for 2026. https://monday.com/blog/rnd/ai-for-software-engineering/
15. [P] ProdMap. (2026). Writing PRDs for AI agents: The new engineering standard. https://www.prodmap.ai/blog/prds-for-ai-agents/
16. [P] Replit. (2026). Vibe coding tools guide: Best AI app builders 2026. https://replit.com/discover/best-vibe-coding-tools
17. [P] Snowflake Engineering. (2026). Paradigm shifts of the developer mindset in the age of AI. https://www.snowflake.com/en/engineering-blog/developer-mindset-paradigm-shifts/
18. [P] Tahir. (2026). What is vibe coding? The six levels of AI development. Medium. https://medium.com/@tahirbalarabe2/what-is-vibe-coding-the-six-levels-of-ai-development-27ad8284fcfb
19. [P] TechJack Solutions. (2026). Prompt engineering library 2026: Ultimate free resource. https://techjacksolutions.com/prompt-engineering-library/
20. [P] tianpan.co. (2026). Traditional code review misses AI-generated vulnerabilities — we built an AI-specific security checklist. https://tianpan.co/forum/t/traditional-code-review-misses-ai-generated-vulnerabilities-we-built-an-ai-specific-security-checklist/3277
21. [P] Wikipedia contributors. (2026, March). Vibe coding. Wikipedia. https://en.wikipedia.org/wiki/Vibe_coding

PART D — VERSION 2.1 OPERATIONAL AMENDMENTS

Empirically Derived Revisions from Production Cycle Retrospectives

Version 2.1 is the first revision of Cronos grounded in longitudinal production evidence rather than design reasoning. Between May and June 2026, four full Cronos cycles were executed on production systems in a regulated (SOC 2 / HIPAA-adjacent) SaaS environment, each closing with the structured retrospective defined in Section A.6. The retrospectives of cycles 0001, 0002, 0003, and 0009 — plus one unfilled retrospective left by cycle 0004, itself treated as negative evidence — were cross-analysed for recurring failure modes and validated controls. Every amendment below cites the cycle evidence that motivated it. Parts A–C of this document remain the v2.0 text; Part D supersedes them where they conflict.

D.1 The Three-Role Model

Production practice converged on a simpler role structure than the four-role hierarchy of Part B. The Solution Owner and Technical Product Owner responsibilities are merged into a single **PM** role; the Developer is renamed **Implementer**; the Peer Reviewer is renamed **Validator**. The two quality gates are defined against these roles: **Gate 1** — the plan-approval and re-approval gate, armed by the PM; **Gate 2** — independent validation, requiring Validator ≠ Implementer. The Team Size Scaling Model (A.1) maps unchanged: Tier 1 collapses all three roles into one person; Tier 4 assigns a PM per product area.

D.2 Calendar-Independent Rhythm (D1–D5)

The weekday-bound rhythm of Section III is replaced by relative cycle days. A cycle is exactly five working days, **D1 through D5**, and may begin on any working day. Weekends and public holidays pause the D-clock: a cycle initialised on a Thursday runs Thu (D1), Fri (D2), Mon (D3), Tue (D4), Wed (D5). All ceremonies keep their position — Path Sync on D2 AM, Checkpoint & QA Sync on D4 AM, Demo & Review on D5 Midday — and the Reset Trigger conditions, being duration-based, are unaffected. The fixed five-day length remains the framework's core determinism control: calendar independence is not licence to compress or stretch a cycle.

D.3 The Cycle Chaining Limit and the Technical Health Cycle

The Focus Mandate blocks the Implementer's calendar completely for a cycle. Chained across many consecutive weeks, this produces two documented costs: practitioner fatigue, and progressive disconnection from the wider team — no informal knowledge flow, no cross-team context. The retrospective record shows a third, structural cost: remediation work deferred to a 'Technical Health Cycle' that never gets scheduled. The frontend end-to-end testing gap was flagged in cycle 0001, deferred in 0002, and remained open in 0003.

Mandate — Cycle Chaining Limit

- No person runs more than **three consecutive delivery cycles** in the same role.
- The following week is mandatory non-delivery time: a **Technical Health Cycle** or unblocked calendar.
- The Technical Health Cycle is the scheduled owner of deferred-debt items from prior retrospectives — it is not optional recovery time that can be traded for another delivery cycle.
- Switching to the Validator role resets the chain (validation is a half-day commitment, not a blocked week).
- A fourth consecutive delivery cycle requires an explicit, written, time-bounded exception from the PM.
- The limit counts cycles, not calendar weeks — weekend-straddling starts (D.2) do not reset it.

D.4 The Daily Pulse

Between the D2 Path Sync and the D4 Checkpoint the Implementer previously had no team contact, and the PM's only mid-cycle visibility was event-driven — a Reset Trigger firing after its threshold was already crossed. The Daily Pulse is a three-line asynchronous update posted by the Implementer at the end of every cycle day: **Landed** (what reached a working state — commits, not intentions), **Next** (the single next step), and **Friction** (anything looping or ambiguous, before it crosses a Reset Trigger threshold). It is explicitly not a standup — no scheduled time, no attendance, no discussion. Cronos deliberately replaced calendar-driven escalation with event-driven Reset Triggers, and that trade is preserved. The Pulse adds a **soft trigger**: the same Friction line in two consecutive Pulses prompts a PM check-in, catching prompt-loop stagnation from outside before the 4-hour hard trigger fires from inside. It simultaneously mitigates the disconnection cost that motivates D.3: the chaining limit caps the damage; the Pulse reduces the per-cycle dose.

D.5 Gate 1 Refinements: Amendment Taxonomy and the Plan Digest

Cycle 0009 grew its plan through six dated amendments, each mechanically blocking code until the human re-approved — the gate held under real pressure. It also exposed two frictions: the amendment counter could not distinguish a material scope change from a verification-phase fix, forcing needless re-approvals; and the plan grew to ~32.5 KB, a real re-orientation cost for every fresh agent session. Two refinements follow.

Amendment taxonomy. Every dated plan amendment is classified as **Material** (new scope, changed design, superseded decision — blocks code until the PM re-approves), **Verification-fix** (an in-scope fix emerging from the verification phase — logged, no re-arm), or **Cosmetic** (logged, no re-arm). The Implementer proposes the class; the Validator may contest it. Misclassifying a Material amendment as Cosmetic is a gate violation.

Plan digest. When the PRD exceeds approximately 20 KB, the Implementer produces a digest of at most 2 KB — current state, active work item, open amendments — which becomes the agent's session-start context. Completed work items collapse to one-line entries with commit pointers.

D.6 Gate 2 Hardening: Mandatory Independent Validation and the Fresh-Eyes Audit

The evidence for Gate 2 is now empirical rather than argued. The one cycle that ran solo on a high-risk tier (0001) shipped its two riskiest changes — an unauthenticated webhook and an expanded delete rule — without adversarial testing, and carried the cost as open follow-ups through the two subsequent cycles.

The cycles that held the gate (0002, 0009) both identified it in retrospect as their highest-value control; in 0009 the Validator's cold run covered 310 tests across two repositories and approved with no conditions. Accordingly: any cycle at **Medium risk or above must name a Validator distinct from the Implementer at D1**. A solo cycle at that tier is a framework deviation requiring written sign-off from the release authority and a stated compensating control.

The fresh-eyes audit. Between 'tests pass' and the D5 demo, a fresh agent context (or fresh human) audits an explicit file list against a severity rubric (critical / major / minor / clean). In cycle 0002 this step caught two dormant endpoints, a silently defaulting parameter, and a partial-save failure mode — all invisible to the author. The same cycle also demonstrated the counter-rule: one audit finding asserted a bug that did not exist. **Audit findings are claims, not facts** — each must be verified against the actual code before being fixed or dismissed.

D.7 Verification Standards

Five standards, each traceable to a specific production failure or near-miss, are added to the verification phase and to the agent guardrail files (.cursorrules):

Standard	Rule	Evidence
Proven-red regression tests	A regression test must be observed failing on the buggy code before it is trusted; the verification record shows the red run.	Validated independently in cycles 0002 and 0009.
Explicit type-check gate	Verification commands run the compiler check explicitly. A suite-level test failure with zero failing assertions is treated as a compile/load error and confirmed in-band.	0009: a parallel test runner silently dropped a suite with a compile error, reporting a passing build.
Loud-fail parameters	Required parameters throw on missing input; silent defaults are banned.	0002: a silent environment default created a misrouting footgun.
Just-in-time construction	Endpoints, services, and public methods are built when their first caller exists — never scaffolded because the specification lists them.	0002: two spec-scaffolded endpoints were built, never called, and deleted mid-cycle.
Narrow-scope formatting	Agents format only the paths actually edited; repository-wide format aliases are banned.	Recurred in 0001 (~60 unrelated files) and again in 0002 despite the written lesson.

A sixth, planning-phase standard: **sibling-repository precedence**. Before designing an integration with an external system, search sibling repositories for a working integration against that same system. In cycle 0009 this check reversed a research-based integration design before any code was written — proven in-organisation code outranks first-principles research.

D.8 Release Governance: Deployment Artefacts, Four Release States, and the Retro Gate

Deployment artefacts are release-critical. Cycle 0003 was held because security-rule deployment was treated as a post-merge operational task, leaving a live risk window. Security rules, IAM grants, bucket lifecycle rules, environment configuration, and feature flags are enumerated in the PRD and verified deployed — or explicitly gated — before a release decision of Shipped. Assumed infrastructure safety nets must be confirmed to exist, not assumed.

Four release states. The binary ship/hold decision proved too coarse: cycle 0001 annotated its own decision as "shipped with conditions would be more accurate", and 0009 shipped to staging with production pending. The release decision now takes one of four states: **Shipped** (production, unconditional); **Shipped with conditions** (deployed, possibly staging-only, with named conditions tracked as follow-ups — the decision reopens if a condition surfaces a real problem); **Held** (with a named unblock condition); **Rolled back** (with reason).

The retrospective is a close gate. One production cycle left its retrospective as an unfilled template — the exact artefact Section A.6 identifies as the primary vehicle of cross-cycle learning. A cycle is not closed, and learning promotion does not run, until the retrospective is completed by all three roles, or carries an explicit "uneventful — nothing to report" entry per role. An uneventful cycle recorded explicitly is signal; an empty template is a process failure.

D.9 The Recurrence Rule

The single most consequential empirical finding of the v2.1 evidence base is that **written lessons do not propagate on their own**. The formatter-overreach lesson was written in cycle 0001's retrospective and violated again in cycle 0002 by a team that had read it. Therefore: any lesson appearing in two consecutive retrospectives must be converted into an enforced mechanism — a guardrail entry in .cursorrules, a CI check, or an automated skill — in the following cycle. A twice-written lesson is reclassified from knowledge to process bug. This rule operationalises the Hallucination Log principle of Section A.6 at the process level: mitigations belong in machinery, not memory.

D.10 Summary of v2.0 → v2.1 Changes

Area	v2.0	v2.1
Roles	SO, TPO, Developer, Peer Reviewer	PM, Implementer, Validator (Gate 1 / Gate 2 defined against roles)
Rhythm	Monday–Friday, calendar-bound	D1–D5 relative days; any start day; weekends pause the clock
Cycle chaining	Unlimited; Technical Health Cycle 'encouraged' every 4th–5th cycle	Hard limit: max 3 consecutive delivery cycles; Technical Health Cycle mandatory and scheduled
Daily visibility	None between syncs	Daily Pulse (async, three lines) + soft Reset Trigger
Plan amendments	Undifferentiated	Material / Verification-fix / Cosmetic taxonomy; only Material re-arms Gate 1
Independent validation	Mandated in principle	Mandatory at Medium+ risk; deviation requires written exception; fresh-eyes audit added
Verification	Tests, scans, visual audits	+ proven-red regression tests, explicit type-check gate, loud-fail parameters, JIT construction, narrow formatting
Release decision	Ship / hold	Four states incl. Shipped with conditions; deployment artefacts release-critical; retro is a close gate
Lesson propagation	Retrospective + Hallucination Log	+ Recurrence Rule: twice-written lessons must be mechanised

The reference implementation of these amendments — templates (agent-ready PRD with amendment log, D5 validation checklist, Daily Pulse), guardrail rule blocks, and the full amendment rationale with per-cycle

citations — is maintained in the framework repository under templates/, registry/cursorrules/, and doc/v2.1-amendments.md.